

Nesta etapa da pesquisa, foram utilizadas as seguintes bibliotecas *Python* para implementar o pipeline de detecção de contas automatizadas (*bots*):

- **Pandas (pandas)**: Biblioteca fundamental para manipulação e análise de dados. Foi utilizada para carregar os datasets, realizar transformações, filtrar dados, criar novas colunas e agregações por usuário.
- **NumPy (numpy)**: Utilizada para operações numéricas eficientes, como cálculos matemáticos e manipulação de arrays. Foi empregada em operações estatísticas e implementação do decaimento exponencial para o fator de confiabilidade.
- **Re (re)**: Biblioteca nativa para buscas e substituições de texto com expressões regulares. Foi aplicada na identificação de padrões em *usernames* para detecção de números aleatórios.
- **Botometer (botometer)**: Biblioteca para integração com a API do *Botometer X*. Utilizada para obter scores de automação baseados em *machine learning* para as contas analisadas.
- **Matplotlib (matplotlib.pyplot)**: Biblioteca para criação de visualizações e gráficos. Foi usada para gerar histogramas, séries temporais e gráficos de barras dos resultados.
- **Datetime (datetime)**: Biblioteca nativa para manipulação de datas e horários. Empregada no cálculo de diferenças temporais, idade das contas e processamento de timestamps.
- **Seaborn (seaborn)**: Biblioteca para visualizações estatísticas avançadas. Utilizada para criar distribuições com KDE e melhorar a estética dos gráficos.
- **Matplotlib.dates (matplotlib.dates)**: Módulo especializado para formatação de datas nos eixos temporais dos gráficos.
- **Time (time)**: Biblioteca nativa para controle de tempo. Foi usada para implementar pausas entre requisições em lote à API do Botometer.

Importando Bibliotecas e Carregando Dataset:

```
[1]: !pip install botometer
```

```
Collecting botometer
Downloading botometer-2.0.1-py3-none-any.whl.metadata (277 bytes)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from botometer) (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->botometer) (3.4.3)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->botometer) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->botometer) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->botometer) (2025.8.3)
Downloading botometer-2.0.1-py3-none-any.whl (3.2 kB)
```

Installing collected packages: botometer
Successfully installed botometer-2.0.1

```
[2]: import pandas as pd
import re
import numpy as np
import time
import botometer
import matplotlib.pyplot as plt
import datetime
import seaborn as sns
import matplotlib.dates as mdates
```

```
[3]: # Carreagando dataset
df = pd.read_csv('/content/drive/MyDrive/TCC/Data/TUEBR22.csv',
sep=",", low_memory=False,
parse_dates=['tweet_created_at', 'user_created_at', 'collection_date'])
```

```
[4]: # Carreagando dataset de mapeamento dos usuários
df_users = pd.read_csv('/content/drive/MyDrive/TCC/Data/mapeamento_usuarios.
↵CSV',
sep=",", low_memory=False)
```

NOTA: SERÃO CLASSIFICADOS PARA ANÁLISE APENAS OS USUÁRIOS AUTORES DE TWEETS PRESENTES NO CORPUS. MENÇÕES E RESPOSTAS A USUÁRIOS EXTERNOS NÃO SERÃO INCLUÍDAS NESTA ETAPA.

C.1 Classificação Heurística

Esta classificação identifica contas automatizadas (*bots*) usando 7 indicadores comportamentais com base em padrões típicos de atividade não-humana no *Twitter/X*. Cada indicador gera pontos que compõem o `bot_score` heurístico.

Referências:

DOTTO, C.; CUBBON, S. Detectar un bot en redes sociales: 3 formas efectivas de identificarlos. 2019. Disponível em: <https://firstdraftnews.org/articles/how-to-spot-a-bot-or-not-the-main-indicators-of-online-automation-co-ordination-and-inauthentic-activity/>. Acesso em: 28 mar. 2025

RIDGE, B. V. Detectar un bot en redes sociales: 3 formas efectivas de identificarlos. 2024. Disponível em: <https://www.mediummultimedia.com/social-media/cuales-son-las-3-formas-en-que-puedes-detectar-un-bot/>. Acesso em: 28 mar. 2025

```
[ ]: # Total de usuários
df['user_id'].nunique()
```

```
[ ]: 84693
```

```
[ ]: # Função para avaliar o username
def is_obvious_sequence(s):
    # Detecta sequências numéricas óbvias, repetitivas ou relacionadas a anos
    if len(s) < 4:
        return False

    # Sequência crescente
    if s in '0123456789' * 2:
        return True

    # Sequência decrescente
    if s in '9876543210' * 2:
        return True

    # Todos os dígitos iguais
    if len(set(s)) == 1:
        return True

    # Padrões repetitivos
    if re.match(r'^(\\d\\d)\\1+$', s) or re.match(r'^(\\d\\d\\d)\\1+$', s):
        return True

    # Verifica se é um ano entre 1900 e 2099
    if 1900 <= int(s) <= 2099:
        return True
```

```

    return False

def has_random_numbers(username):

    # Verifica se um username contém padrões de números aleatórios.
    # Extrair todos os grupos de 4+ dígitos
    digit_groups = re.findall(r'\d{4,}', username)
    for group in digit_groups:
        if not is_obvious_sequence(group):
            return True

    # Proporção de dígitos > 50%
    digit_count = sum(c.isdigit() for c in username)
    total_chars = len(username)
    if digit_count >= 5 and digit_count / total_chars > 0.5:
        return True

    return False

```

```

[ ]: # Testando com diferentes padrões
test_cases = [
    "Joao_39472",      # True (Critério 1)
    "Maria_123",       # False (poucos dígitos)
    "User92741",       # True (Critério 2 - sufixo aleatório)
    "1234_User",       # False (números no início)
    "NormalUser",      # False (sem números)
    "XyZ_123",         # False (poucos dígitos)
    "Bt_111",          # False (Critério 3 - dígitos repetidos)
    "user5656",        # False (Critério 4 - sequência interna)
    "john2023",        # False (Critério 5 - ano)
    "rand_4683",       # True (sufixo aleatório)
    "user_24601",      # True (Critério 1)
    "random_9182"      # True (sufixo aleatório)
]

for username in test_cases:
    print(f"{username}: {has_random_numbers(username)}")

```

```

Joao_39472: True
Maria_123: False
User92741: True
1234_User: False
NormalUser: False
XyZ_123: False
Bt_111: False
user5656: False
john2023: False

```

```
rand_4683: True
user_24601: True
random_9182: True
```

```
[ ]: # Aplicando a função
df_users['has_random_numbers'] = df_users['username_original'].
↳apply(has_random_numbers)
```

```
[ ]: # Obtém os IDs pseudônimos únicos de usuários
users_in_df = df['user_id'].unique()

# Filtra df_users para manter apenas linhas onde 'user_pseudo' está na lista de
↳usuários
result_df = df_users[df_users['user_pseudo'].isin(users_in_df)][
    ['user_id_original', 'user_pseudo', 'has_random_numbers']]
```

```
[ ]: # Deriva colunas auxiliares para análise temporal
df['data'] = df['tweet_created_at'].dt.date
df['hora'] = df['tweet_created_at'].dt.hour
df['atividade_madrugada'] = df['hora'].between(2, 5)

# Agregações por usuário
usuarios = df.groupby('user_id').agg(
    total_tweets=('tweet_id', 'count'),
    retweet_ratio=('is_retweet', 'mean'),
    madrugada_ratio=('atividade_madrugada', 'mean'),
    hashtags_media=('hashtags_count', 'mean'),
    account_age_days=('account_age_days', 'first'),
    data_min=('tweet_created_at', 'min'),
    data_max=('tweet_created_at', 'max')
)

# Calcula o limite do top 5% de atividade
limite_top_5 = usuarios['total_tweets'].quantile(0.95)

# Prepara result_df
result_df = result_df[result_df['user_pseudo'].isin(usuarios.index)]
result_df = result_df.drop_duplicates(subset='user_pseudo', keep='first')

# Merge e preenchimento de valores ausentes
usuarios = usuarios.merge(result_df.set_index('user_pseudo'), left_index=True,
↳right_index=True, how='left')
usuarios['has_random_numbers'] = usuarios['has_random_numbers'].fillna(False)

# Usar total absoluto de hashtags
```

```

usuarios['hashtags_total'] = usuarios['hashtags_media'] * \
    usuarios['total_tweets']

def bot_score(row):
    score = 0
    categorias = []

    # 1. Alto volume (top 5%)
    if row['total_tweets'] > limite_top_5:
        score += 1.5
        categorias.append("volume_alto")

    # 2. Alta proporção de retweets
    if row['retweet_ratio'] > 0.8:
        if row['total_tweets'] > 30:
            score += 1.5
            categorias.append("retweet_alto")
        else:
            score += 0.5
            categorias.append("retweet_medio")

    # 3. Padrões irregulares de atividade (coeficiente de variação)
    user_data = df[df['user_id'] == row.name]
    daily_activity = user_data.groupby('data').size()

    if len(daily_activity) > 3: # suficiente para calcular variação
        mean = daily_activity.mean()
        std = daily_activity.std()
        if mean > 0:
            cv = std / mean
            if cv > 1.0:
                score += 1
                categorias.append("atividade_irreg")

    # 4. Conta nova
    if row['account_age_days'] < 15:
        score += 1
        categorias.append("nova_15d")
    elif row['account_age_days'] < 30:
        score += 0.5
        categorias.append("nova_30d")

    # 5. Muitas hashtags
    if row['hashtags_total'] > 50:
        score += 1
        categorias.append("hashtags_excesso")

```

```

# 6. Username com números aleatórios
if row['has_random_numbers']:
    score += 0.5
    categorias.append("username_num")

# 7. Atividade concentrada na madrugada
if row['madrugada_ratio'] > 0.5:
    score += 0.5
    categorias.append("atividade_madruga")

return pd.Series({'bot_score': score, 'categorias_pontuadas': categorias})

# Aplica a função
usuarios[['bot_score', 'categorias_pontuadas']] = usuarios.apply(bot_score,
↪axis=1)

# E para merge final:
df = df.merge(
    usuarios[['bot_score', 'categorias_pontuadas']],
    on='user_id',
    how='left'
)

```

```

[ ]: # Reiniciando índice
usuarios = usuarios.rename_axis('user_id').reset_index()

```

```

[ ]: # Verificando se todos usuários foram classificados
usuarios['user_id'].unique().size

```

```

[ ]: 84693

```

```

[ ]: # Função para converter a escala de classificação para variar de 0 a 1
# Com objetivo de facilitar a comparação com Botometer
def converter_escala(valor, casas=2):
    return round(valor / 7, casas)

```

```

[ ]: # Aplicando função
usuarios['bot_score'] = usuarios['bot_score'].apply(converter_escala)

```

```

[ ]: usuarios['bot_score'].max()

```

```

[ ]: 0.57

```

```

[ ]: usuarios['bot_score'].value_counts().sort_index()

```

```

[ ]: bot_score
0.00    26173

```

```
0.07    41060
0.14    11793
0.21     3266
0.29     1862
0.36      415
0.43       97
0.50       24
0.57        3
Name: count, dtype: int64
```

```
[ ]: # Salvando scores
      usuarios.to_csv("/content/drive/MyDrive/TCC/Data/bot_score.csv", index=False)
```

C.2 Classificação via Botometer X

C.2.1 Função para Acessar API do Botometer X

Devido a limitações recentes no uso da API do *Twitter/X* (encerramento da API gratuita em 31/05/2023), a versão atual do *Botometer*, a *Botometer X*, opera exclusivamente em modo de arquivamento com as seguintes características:

Restrições Fundamentais

Janela temporal fixa

- Dados históricos congelados (atividade até maio de 2023)
- Contas criadas após 31/05/2023 **não são analisáveis**
- Atualizações de perfil pós-limite não refletidas

Restrições operacionais

- O ponto final do Botometer X está disponível em todos os planos
- **Plano gratuito:** 50 solicitações/mês ($50 \times 100 = 5.000$ contas)
- **Plano PRO:** 50 solicitações/dia gratuitamente; excedente cobrado a US\$ 1/100 solicitações.
 - **Plano ULTRA:** US\$ 50/mês, com 250 solicitações/dia; excedente cobrado a US\$ 1/100 solicitações
 - Máximo de 100 contas por requisição
 - Necessidade de desduplicação manual quando enviados `user_id` + `username`

Estrutura da Resposta

A resposta será uma lista de objetos JSON. Significados dos elementos no objeto:

- `bot_score`: Pontuação do bot (número entre 0 e 1; redimensionado para 1 a 5 no site do Botometer X)
- `timestamp`: Hora em que a pontuação foi calculada
- `user_id`: ID da conta
- `username`: Nome de usuário da conta

Referências: <https://rapidapi.com/OSoMe/api/botometer-pro>

```
[ ]: # Função que busca em Lote
def get_bot_scores(user_ids):

    # Configuração da API
    rapidapi_key = "#####"

    bomx = botometer.BotometerX(rapidapi_key=rapidapi_key)

    # Resultado acumulado
```

```

resultados = []

# Processar em lotes de até 100
batch_size = 100
for i in range(0, len(user_ids), batch_size):
    batch_ids = user_ids[i:i+batch_size]

    try:
        print(f"🔍 Processando {len(batch_ids)} contas (lote {i//
↪batch_size + 1})...")
        data = bomx.get_bot_scores_in_batch(user_ids=batch_ids)

        # Se a API já retorna os dados do lote, não precisa fatiar
        response = {"results": data}

        for result in response.get("results", []):
            if isinstance(result, dict):
                resultados.append({
                    "user_id": result.get("user_id"),
                    "username": result.get("username"),
                    "bot_score": result.get("bot_score"),
                    "timestamp": result.get("timestamp")
                })
            else:
                print(f"⚠️ Formato inesperado no resultado: {result}")

        except Exception as e:
            print(f"❌ Erro no lote {i//batch_size + 1}: {e}")

        time.sleep(2)

    # Salva os resultados em CSV
    resultados = pd.DataFrame(resultados)
    resultados.to_csv("/content/drive/MyDrive/TCC/Data/
↪resultados_botometer_lote.csv", index=False)
    print(f"✅ Resultados salvos em 'resultados_botometer_lote.csv'")

    return resultados

```

C.2.2 Aplicar Função

C.2.2.1 Busca Inicial (Filtragem Básica)

```

[5]: # Carregando dataset com as classificações anteriores
usuarios = pd.read_csv('/content/drive/MyDrive/TCC/Data/bot_score.csv',
                        sep=",", low_memory=False)
usuarios['user_id'].size

```

[5]: 84693

```
[6]: # Filtrando IDs corrompidos
amostra_filtrada = usuarios[usuarios['user_id_original'].str.contains(',', na=False)]
amostra_filtrada['user_id_original'].size
```

[6]: 72

```
[7]: # Remover usuários com ID corrompido
# Converter as colunas para string
todos_usuarios = usuarios.astype({'user_id_original': str})
amostra_filtrada = amostra_filtrada.astype({'user_id_original': str})

# Criar um conjunto com os IDs
ids_amostra = set(amostra_filtrada['user_id_original'])

# Filtrar linhas onde user_id_original NÃO está no conjunto
amostra = todos_usuarios[~todos_usuarios['user_id_original'].isin(ids_amostra)]

amostra['user_id_original'].size
```

[7]: 84621

```
[8]: # Sorteia 5000 (LIMITE DE REQUEST POR DIA DA API DO BOTOMETER X)
# Determina um tamanho de amostra seguro (o menor entre 5000 e o tamanho real do DataFrame)
sample_size = min(5000, len(amostra))

# Realiza a amostragem sem reposição usando o tamanho seguro
amostra = amostra.sample(n=sample_size, random_state=42)
amostra['user_id_original'].size
```

[8]: 5000

```
[ ]: # Buscar na API do Botometer
user_ids = amostra['user_id_original'].dropna().astype(str).unique().tolist()
resultados = get_bot_scores(user_ids)
```

```
🔍 Processando 100 contas (lote 1)...
🔍 Processando 100 contas (lote 2)...
🔍 Processando 100 contas (lote 3)...
🔍 Processando 100 contas (lote 4)...
🔍 Processando 100 contas (lote 5)...
🔍 Processando 100 contas (lote 6)...
🔍 Processando 100 contas (lote 7)...
🔍 Processando 100 contas (lote 8)...
🔍 Processando 100 contas (lote 9)...
```

 Processando 100 contas (lote 10)...

 Processando 100 contas (lote 11)...

 Processando 100 contas (lote 12)...

 Processando 100 contas (lote 13)...

 Processando 100 contas (lote 14)...

 Processando 100 contas (lote 15)...

 Processando 100 contas (lote 16)...

 Processando 100 contas (lote 17)...

 Processando 100 contas (lote 18)...

 Processando 100 contas (lote 19)...

 Processando 100 contas (lote 20)...

 Processando 100 contas (lote 21)...

 Processando 100 contas (lote 22)...

 Processando 100 contas (lote 23)...

 Processando 100 contas (lote 24)...

 Processando 100 contas (lote 25)...

 Processando 100 contas (lote 26)...

 Processando 100 contas (lote 27)...

 Processando 100 contas (lote 28)...

 Processando 100 contas (lote 29)...

 Processando 100 contas (lote 30)...

 Processando 100 contas (lote 31)...

 Processando 100 contas (lote 32)...

 Processando 100 contas (lote 33)...

 Processando 100 contas (lote 34)...

 Processando 100 contas (lote 35)...

 Processando 100 contas (lote 36)...

 Processando 100 contas (lote 37)...

 Processando 100 contas (lote 38)...

 Processando 100 contas (lote 39)...

 Processando 100 contas (lote 40)...

 Processando 100 contas (lote 41)...

 Processando 100 contas (lote 42)...

 Processando 100 contas (lote 43)...

 Processando 100 contas (lote 44)...

 Processando 100 contas (lote 45)...

 Processando 100 contas (lote 46)...

 Processando 100 contas (lote 47)...

 Processando 100 contas (lote 48)...

 Processando 100 contas (lote 49)...

 Processando 100 contas (lote 50)...

 Resultados salvos em 'resultados_botometer_lote.csv'

```
[ ]: resultados['user_id'].size
```

```
[ ]: 4988
```

```
[ ]: # (APENAS SE A BUSCA OCORRER SEM ERROS)
# Filtrando usuários que não foram encontrados
# Converter as colunas para string
amostra = amostra.astype({'user_id_original': str})
resultados = resultados.astype({'user_id': str})

# Criar um conjunto com os IDs
ids = set(resultados['user_id'])

# Filtrar linhas onde user_id_original NÃO está no conjunto de IDs
sem_score = amostra[~amostra['user_id_original'].isin(ids)]

sem_score['user_id_original'].size
```

[]: 12

```
[ ]: # (APENAS SE A BUSCA OCORRER SEM ERROS)
# Salvando usuários que não foram encontrados (para evitar reconsulta)
sem_score.to_csv("/content/drive/MyDrive/TCC/Data/amostra_sem_score.csv",
↳ index=False)
```

```
[ ]: # Salvando
resultados.to_csv("/content/drive/MyDrive/TCC/Data/bot_score_botometer.csv",
↳ index=False)
```

C.2.2.2 Loop de Busca Final (Filtragem Otimizada)

```
[ ]: # Carregando dataset com as classificação anterior
usuarios = pd.read_csv('/content/drive/MyDrive/TCC/Data/bot_score.csv',
    sep=";", low_memory=False)
usuarios['user_id'].size
```

[]: 84693

```
[ ]: # Usuários já possuem score
score_botometer = pd.read_csv('/content/drive/MyDrive/TCC/Data/
↳ bot_score_botometer.csv',
    sep=";", low_memory=False)
score_botometer['user_id'].size
```

[]: 79606

```
[ ]: # Remover usuários que já foram classificados
# Converter as colunas para string
amostra = usuarios.astype({'user_id_original': str})
score_botometer = score_botometer.astype({'user_id': str})
```

```

# Criar um conjunto com os IDs do score_botometer para busca
ids_botometer = set(score_botometer['user_id'])

# Filtrar linhas onde user_id_original NÃO está no conjunto de IDs
amostra_nao_classificada = amostra[~amostra['user_id_original'].
    ↪isin(ids_botometer)]

amostra_nao_classificada['user_id_original'].size

```

[]: 5087

```

[ ]: # Usuários que não contam na base do Botometer X
amostra_sem_score = pd.read_csv('/content/drive/MyDrive/TCC/Data/
    ↪amostra_sem_score.csv',
    sep=",", low_memory=False)
amostra_sem_score['user_id_original'].size

```

[]: 206

```

[ ]: # Remover usuários que não possuem score registrado no Botometer
# Converter as colunas para string
todos_usuarios = amostra_nao_classificada.astype({'user_id_original': str})
amostra_sem_score = amostra_sem_score.astype({'user_id_original': str})

# Criar um conjunto com os IDs
ids_sem_score = set(amostra_sem_score['user_id_original'])

# Filtrar linhas onde user_id_original NÃO está no conjunto de IDs
amostra_sem_invalidos = todos_usuarios[~todos_usuarios['user_id_original'].
    ↪isin(ids_sem_score)]

amostra_sem_invalidos['user_id_original'].size

```

[]: 4881

```

[ ]: # Usuários com IDs corrompidos
usuarios_com_id_corrompido = □
    ↪amostra_sem_invalidos[amostra_sem_invalidos['user_id_original'].str.
    ↪contains(',', na=False)]
usuarios_com_id_corrompido['user_id_original'].size

```

[]: 72

```

[ ]: # Remover usuários com ID corrompido
# Converter as colunas para string
amostra = amostra_sem_invalidos.astype({'user_id_original': str})

```

```

usuarios_com_id_corrompido = usuarios_com_id_corrompido.
    ↳astype({'user_id_original': str})

# Criar um conjunto com os IDs
ids = set(usuarios_com_id_corrompido['user_id_original'])

# Filtrar linhas onde user_id_original NÃO está no conjunto
amostra = amostra[~amostra['user_id_original'].isin(ids)]

amostra['user_id_original'].size

```

[]: 4809

```

[ ]: # Sorteia 5000 (LIMITE DE REQUEST POR DIA DA API DO BOTOMETER X)
# Determina um tamanho de amostra seguro (o menor entre 5000 e o tamanho real
    ↳do DataFrame)
sample_size = min(5000, len(amostra))

# Realiza a amostragem sem reposição usando o tamanho seguro
amostra = amostra.sample(n=sample_size, random_state=42)
amostra['user_id_original'].size

```

[]: 4809

```

[ ]: # Buscar na API do Botometer
user_ids = amostra['user_id_original'].dropna().astype(str).unique().tolist()
resultados = get_bot_scores(user_ids)

```

```

🔍 Processando 100 contas (lote 1)...
🔍 Processando 100 contas (lote 2)...
🔍 Processando 100 contas (lote 3)...
🔍 Processando 100 contas (lote 4)...
🔍 Processando 100 contas (lote 5)...
🔍 Processando 100 contas (lote 6)...
🔍 Processando 100 contas (lote 7)...
🔍 Processando 100 contas (lote 8)...
🔍 Processando 100 contas (lote 9)...
🔍 Processando 100 contas (lote 10)...
🔍 Processando 100 contas (lote 11)...
🔍 Processando 100 contas (lote 12)...
🔍 Processando 100 contas (lote 13)...
🔍 Processando 100 contas (lote 14)...
🔍 Processando 100 contas (lote 15)...
🔍 Processando 100 contas (lote 16)...
🔍 Processando 100 contas (lote 17)...
🔍 Processando 100 contas (lote 18)...
🔍 Processando 100 contas (lote 19)...
🔍 Processando 100 contas (lote 20)...

```

🔍 Processando 100 contas (lote 21)...

🔍 Processando 100 contas (lote 22)...

🔍 Processando 100 contas (lote 23)...

🔍 Processando 100 contas (lote 24)...

🔍 Processando 100 contas (lote 25)...

🔍 Processando 100 contas (lote 26)...

🔍 Processando 100 contas (lote 27)...

🔍 Processando 100 contas (lote 28)...

🔍 Processando 100 contas (lote 29)...

🔍 Processando 100 contas (lote 30)...

🔍 Processando 100 contas (lote 31)...

🔍 Processando 100 contas (lote 32)...

🔍 Processando 100 contas (lote 33)...

🔍 Processando 100 contas (lote 34)...

🔍 Processando 100 contas (lote 35)...

🔍 Processando 100 contas (lote 36)...

🔍 Processando 100 contas (lote 37)...

🔍 Processando 100 contas (lote 38)...

🔍 Processando 100 contas (lote 39)...

🔍 Processando 100 contas (lote 40)...

🔍 Processando 100 contas (lote 41)...

🔍 Processando 100 contas (lote 42)...

🔍 Processando 100 contas (lote 43)...

🔍 Processando 100 contas (lote 44)...

🔍 Processando 100 contas (lote 45)...

🔍 Processando 100 contas (lote 46)...

🔍 Processando 100 contas (lote 47)...

🔍 Processando 100 contas (lote 48)...

🔍 Processando 9 contas (lote 49)...

OK Resultados salvos em 'resultados_botometer_lote.csv'

```
[ ]: resultados['user_id'].size
```

```
[ ]: 4809
```

```
[ ]: # (APENAS SE A BUSCA OCORRER SEM ERROS)
# Filtrando usuários que não foram encontrados
# Converter as colunas para string
amostra = amostra.astype({'user_id_original': str})
resultados = resultados.astype({'user_id': str})

# Criar um conjunto com os IDs
ids = set(resultados['user_id'])

# Filtrar linhas onde user_id_original NÃO está no conjunto de IDs
sem_score = amostra[~amostra['user_id_original'].isin(ids)]

sem_score['user_id_original'].size
```

```
[ ]: 0
```

```
[ ]: # Concatenado dataframes
frames = [sem_score, amostra_sem_score]
sem_score = pd.concat(frames)
sem_score.reset_index(drop=True)
sem_score['user_id'].size
```

```
[ ]: 206
```

```
[ ]: # (APENAS SE A BUSCA OCORRER SEM ERROS)
# Salvando usuários que não foram encontrados (para evitar reconsulta)
sem_score.to_csv("/content/drive/MyDrive/TCC/Data/amostra_sem_score.csv",
    ↪index=False)
```

```
[ ]: # Usuários já possuem score
score_botometer = pd.read_csv('/content/drive/MyDrive/TCC/Data/
    ↪bot_score_botometer.csv',
                                sep=",", low_memory=False)
score_botometer['user_id'].size
```

```
[ ]: 79606
```

```
[ ]: # Concatenar dataframes (Nova busca + Antiga)
frames = [score_botometer, resultados]
df0 = pd.concat(frames)
df0.reset_index(drop=True)
df0['user_id'].size
```

```
[ ]: 84415
```

```
[ ]: # Verificar se há dados duplicados
duplicatas = df0[df0.duplicated('user_id', keep=False)]
duplicatas['user_id'].size
```

```
[ ]: 0
```

```
[ ]: # Salvando
df0.to_csv("/content/drive/MyDrive/TCC/Data/bot_score_botometer.csv",
    ↪index=False)
```

C.2.2.3 Buscar de Usuários com IDs Corrompidos pelo Username

```
[ ]: # Carregando dataset com as classificação anterior
usuarios = pd.read_csv('/content/drive/MyDrive/TCC/Data/bot_score.csv',
                        sep=",", low_memory=False)
usuarios['user_id'].size
```

[]: 84693

```
[ ]: # Usuários já possuem score
score_botometer = pd.read_csv('/content/drive/MyDrive/TCC/Data/
    ↳bot_score_botometer.csv',
                                sep=";", low_memory=False)
score_botometer['user_id'].size
```

[]: 84415

```
[ ]: # Usuários que não contam na base do Botometer X
amostra_sem_score = pd.read_csv('/content/drive/MyDrive/TCC/Data/
    ↳amostra_sem_score.csv',
                                sep=";", low_memory=False)
amostra_sem_score['user_id_original'].size
```

[]: 206

```
[ ]: # Remover usuários que já foram classificados
# Converter as colunas para string
amostra = usuarios.astype({'user_id_original': str})
score_botometer = score_botometer.astype({'user_id': str})

# Criar um conjunto com os IDs do score_botometer para busca
ids_botometer = set(score_botometer['user_id'])

# Filtrar linhas onde user_id_original NÃO está no conjunto de IDs
amostra_nao_classificada = amostra[~amostra['user_id_original'].
    ↳isin(ids_botometer)]

amostra_nao_classificada['user_id_original'].size
```

[]: 278

```
[ ]: # Usuários com IDs corrompidos
usuarios_com_id_corrompido = amostra_nao_classificada[
    amostra_nao_classificada['user_id_original'].astype(str).str.contains(',',,
    ↳na=False)]

usuarios_com_id_corrompido['user_id_original'].size
```

[]: 72

```
[ ]: # Obter username original
amostra = df_users[
df_users['user_id_original'].
    ↳isin(usuarios_com_id_corrompido['user_id_original'])].copy()
```

```
amostra['user_id_original'].size
```

```
[ ]: 72
```

```
[ ]: # Removendo '@' do username
amostra["username_original"] = amostra["username_original"].str.replace(r"^@", "\u
↪", regex=True)
```

```
[ ]: # Extraíndo os dígitos mais significativos do User ID corrompidos
amostra["valor_significativo"] = amostra["user_id_original"].apply(
    lambda x: str(x).split("E")[0].replace(",", "").replace(".", "")
)
```

```
[ ]: # Buscar na API do Botometer pelo username

# Lista de usernames
usernames = amostra['username_original'].dropna().astype(str).unique().tolist()

# Chave da API
rapidapi_key = "#####"
```

```
# Inicializar o BotometerX
bomx = botometer.BotometerX(rapidapi_key=rapidapi_key)

resultados = []

try:
    # Obter os scores em lote
    data = bomx.get_bot_scores_in_batch(usernames=usernames)

    for result in data:
        try:
            # Verifica se o resultado é um dicionário válido
            if isinstance(result, dict):
                resultados.append({
                    "user_id": result.get("user_id", None),
                    "username": result.get("username", None),
                    "bot_score": result.get("bot_score", None),
                    "timestamp": result.get("timestamp", None)
                })
            else:
                print(f"⚠ Formato inesperado: {result}")
        except Exception as e:
            print(f"❌ Erro ao processar resultado individual: {e}")

except botometer.BotometerException as e:
```

```

    print(f"❌ Erro na API do Botometer: {e}")
except Exception as e:
    print(f"❌ Erro geral: {e}")

# Converter resultados válidos em DataFrame
if resultados:
    resultados = pd.DataFrame(resultados)
    print("✅ Dados processados com sucesso!")
else:
    resultados = pd.DataFrame(columns=["user_id", "username", "bot_score", "
↳timestamp"])
    print("⚠️ Nenhum resultado válido foi retornado.")

```

✅ Dados processados com sucesso!

```
[ ]: resultados["username"].size
```

```
[ ]: 73
```

```
[ ]: # Colocar username em 'lower case' para comparar
resultados["username"] = resultados["username"].str.lower()
score_botometer["username"] = score_botometer["username"].str.lower()

```

```
[ ]: merge = pd.merge(resultados, amostra, left_on='username',
↳right_on='username_original', how='left')

```

```
[ ]: filtr_invalidos = merge[
    merge.apply(lambda row: str(row["user_id"]).
↳startswith(row["valor_significativo"][:3]), axis=1)
]
filtr_invalidos['user_id'].size

```

```
[ ]: 70
```

```
[ ]: users_classificados = filtr_invalidos[
    filtr_invalidos['username_original'].isin(filtr_invalidos['username'])]

users_classificados['user_id'].size

```

```
[ ]: 70
```

```
[ ]: # Selecionando as features corretas
users_classificados = users_classificados[
    ['user_id_original', 'username', 'bot_score', 'timestamp']].copy()
users_classificados['user_id_original'].size

```

```
[ ]: 70
```

```
[ ]: # Renomeando features
users_classificados.rename(columns={"user_id_original": "user_id", "bot_score": "bot_score_botometer"}, inplace=True)
```

```
[ ]: # Concatenar dataframes
frames = [score_botometer, users_classificados]
df0 = pd.concat(frames)
df0.reset_index(drop=True)
df0['user_id'].size
```

```
[ ]: 84485
```

```
[ ]: # Verificar se há dados duplicados
duplicatas = df0[df0.duplicated('user_id', keep=False)]
duplicatas['user_id'].size
```

```
[ ]: 0
```

```
[ ]: # Salvando
df0.to_csv("/content/drive/MyDrive/TCC/Data/bot_score_botometer.csv",
           index=False)
```

C.3 Gerar Score Final

C.3.1 Unificando Datasets

```
[ ]: # Carreando dataset com scores do Heurística
score_heuristica = pd.read_csv('/content/drive/MyDrive/TCC/Data/bot_score.csv',
                               sep=",", low_memory=False)
score_heuristica['user_id'].size
```

```
[ ]: 84693
```

```
[ ]: # Renomeando feature
score_heuristica.rename(columns={"bot_score": "bot_score_heuristica"},
                        inplace=True)
```

```
[ ]: # Carreando dataset com scores do Botometer X
score_botometer = pd.read_csv('/content/drive/MyDrive/TCC/Data/
                               bot_score_botometer.csv',
                               sep=",", low_memory=False)
score_botometer['user_id'].size
```

```
[ ]: 84485
```

```
[ ]: # Renomeando feature
score_botometer.rename(columns={"bot_score": "bot_score_botometer"},
                       inplace=True)
```

```
[ ]: score_botometer[['bot_score_botometer', 'timestamp']]
```

```
[ ]:      bot_score_botometer      timestamp
0          0.06  Thu, 06 Oct 2022 07:59:26 GMT
1          0.37  Sun, 01 Jan 2023 14:38:11 GMT
2          0.23  Sun, 07 May 2023 01:12:42 GMT
3          0.39  Sat, 27 May 2023 02:20:14 GMT
4          0.66  Tue, 28 Feb 2023 00:00:02 GMT
...          ...
84480        0.23  Thu, 01 Jun 2023 02:26:16 GMT
84481        0.10  Sun, 08 Jan 2023 19:30:27 GMT
84482        0.42  Wed, 31 May 2023 09:25:54 GMT
84483        0.05  Thu, 01 Jun 2023 02:18:33 GMT
84484        0.12  Thu, 01 Jun 2023 04:09:54 GMT
```

```
[84485 rows x 2 columns]
```

```
[ ]: # Cria uma nova coluna 'date_calculated' copiando os valores de 'timestamp'
score_botometer['date_calculated'] = score_botometer['timestamp']

# Formata para "AAAA-MM-DD HH:MM:SS"
```

```
score_botometer['date_calculated'] = pd.
↳to_datetime(score_botometer['date_calculated'])
```

```
[ ]: # Validando
score_botometer[['bot_score_botometer', 'timestamp', 'date_calculated']]
```

```
[ ]:
      bot_score_botometer      timestamp      date_calculated
0          0.06  Thu, 06 Oct 2022 07:59:26 GMT 2022-10-06 07:59:26
1          0.37  Sun, 01 Jan 2023 14:38:11 GMT 2023-01-01 14:38:11
2          0.23  Sun, 07 May 2023 01:12:42 GMT 2023-05-07 01:12:42
3          0.39  Sat, 27 May 2023 02:20:14 GMT 2023-05-27 02:20:14
4          0.66  Tue, 28 Feb 2023 00:00:02 GMT 2023-02-28 00:00:02
...
84480          0.23  Thu, 01 Jun 2023 02:26:16 GMT 2023-06-01 02:26:16
84481          0.10  Sun, 08 Jan 2023 19:30:27 GMT 2023-01-08 19:30:27
84482          0.42  Wed, 31 May 2023 09:25:54 GMT 2023-05-31 09:25:54
84483          0.05  Thu, 01 Jun 2023 02:18:33 GMT 2023-06-01 02:18:33
84484          0.12  Thu, 01 Jun 2023 04:09:54 GMT 2023-06-01 04:09:54
```

[84485 rows x 3 columns]

```
[ ]: # Unindo datasets de scores
score_heuristica['user_id_original'] = score_heuristica['user_id_original'].
↳astype(str)
score_botometer['user_id'] = score_botometer['user_id'].astype(str)

df_merged = pd.merge(score_heuristica, score_botometer,
↳left_on='user_id_original', right_on='user_id', how='left')
df_merged[['user_id_x', 'bot_score_heuristica', 'bot_score_botometer', 'date_calculated']]
```

```
[ ]:
      user_id_x  bot_score_heuristica  bot_score_botometer \
0  @USER_0000BA7CDAC2          0.14          0.08
1  @USER_0002491569D2          0.07          0.24
2  @USER_00024B2885C4          0.14          0.10
3  @USER_0002A891974C          0.07          0.28
4  @USER_0002C9F3023B          0.07          0.63
...
84688  @USER_FFFB8012E37F          0.07          0.46
84689  @USER_FFFC7FDD143C          0.07          0.21
84690  @USER_FFFCD1E461F3          0.00          0.53
84691  @USER_FFFCD312E941          0.14          0.36
84692  @USER_FFFD8D93D15C          0.14          0.29

      date_calculated
0  2023-01-05 00:26:48
1  2023-05-31 03:32:32
2  2022-11-08 13:18:27
```

```

3      2023-05-30 01:22:12
4      2023-05-21 20:48:19
...
84688 2023-02-01 17:59:55
84689 2023-03-15 00:13:08
84690 2023-05-30 21:41:57
84691 2022-12-05 16:37:45
84692 2023-04-05 15:35:28

```

[84693 rows x 4 columns]

```
[ ]: # Validando dados
USER = ['#####', '*****', '@@@@@@@@@@@@@@']
```

```
[ ]: score_heuristica[score_heuristica['user_id_original'].isin(USER)
][['user_id', 'bot_score_heuristica']]
```

```
[ ]:
      user_id  bot_score_heuristica
18129  @USER_36D1DF013CDC           0.07
38838  @USER_753F02F3CB8A           0.14
64577  @USER_C290D2DDF85D           0.07
```

```
[ ]: score_botometer[score_botometer['user_id'].isin(USER)
][['bot_score_botometer', 'date_calculated']]
```

```
[ ]:
      bot_score_botometer  date_calculated
2                0.23 2023-05-07 01:12:42
17486              0.12 2023-06-01 02:20:01
84414              0.31 2023-05-25 14:47:06
```

```
[ ]: df_merged[df_merged['user_id_original'].isin(USER)
][['user_id_x', 'bot_score_heuristica', 'bot_score_botometer', 'date_calculated']]
```

```
[ ]:
      user_id_x  bot_score_heuristica  bot_score_botometer \
18129  @USER_36D1DF013CDC           0.07                0.12
38838  @USER_753F02F3CB8A           0.14                0.31
64577  @USER_C290D2DDF85D           0.07                0.23

      date_calculated
18129 2023-06-01 02:20:01
38838 2023-05-25 14:47:06
64577 2023-05-07 01:12:42
```

```
[ ]: # Usuários não encontrados na API do Botometer X (Deve ser 208)
df_conf = df_merged[df_merged['bot_score_botometer'].isna()]
df_conf
```

```
[ ]:      user_id_x  bot_score_heuristica  bot_score_botometer  \
148      @USER_00623796424A              0.21              NaN
406      @USER_01346B26CB5D              0.14              NaN
447      @USER_0150F900312D              0.21              NaN
563      @USER_01A479AB54E9              0.21              NaN
724      @USER_021811D611FF              0.07              NaN
...
82005    @USER_F7D0A0CDCD83              0.14              NaN
82247    @USER_F87C3774102C              0.00              NaN
82615    @USER_F99A223662A0              0.07              NaN
82885    @USER_FA802F724DF2              0.00              NaN
83151    @USER_FB5564032847              0.21              NaN

      date_calculated
148              NaT
406              NaT
447              NaT
563              NaT
724              NaT
...
82005            NaT
82247            NaT
82615            NaT
82885            NaT
83151            NaT

[208 rows x 4 columns]
```

```
[ ]: # Seleccionando columnas importantes
df_merged = df_merged[['user_id_x', 'bot_score_heuristica',
                        'bot_score_botometer', 'date_calculated']]

# Renombrando columna
df_merged.rename(columns={"user_id_x": "user_id"}, inplace= True)
```

```
[ ]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 111217 entries, 0 to 111216
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   tweet_id              111217 non-null object
1   raw_tweet             111217 non-null object
2   preprocessed_tweet    111193 non-null object
3   urls                  17025 non-null  object
4   hashtags              111217 non-null object
5   mentions              111217 non-null object
```

```

6   emojis                111217 non-null  object
7   is_retweet            111217 non-null  int64
8   original_author       69620 non-null  object
9   is_truncated          111217 non-null  int64
10  is_upper              111217 non-null  int64
11  tweet_length          111217 non-null  int64
12  has_keywords          111217 non-null  int64
13  tweet_created_at      111217 non-null  datetime64[ns]
14  reply_to_tweet_id     28331 non-null  object
15  reply_to_user_id      28535 non-null  object
16  retweets              111217 non-null  int64
17  likes                 111217 non-null  int64
18  engagement_raw        111217 non-null  int64
19  engagement_rate        111217 non-null  float64
20  engagement_index      111217 non-null  float64
21  url_count             111217 non-null  int64
22  hashtags_count        111217 non-null  int64
23  mentions_count        111217 non-null  int64
24  emoji_count           111217 non-null  int64
25  user_id               111217 non-null  object
26  followers_count       111217 non-null  int64
27  user_created_at       111217 non-null  datetime64[ns]
28  is_verified           111217 non-null  int64
29  account_age_days      111217 non-null  int64
30  collection_date       111217 non-null  datetime64[ns]
31  is_withheld           111217 non-null  int64
32  electoral_phase       111217 non-null  object
dtypes: datetime64[ns](3), float64(2), int64(16), object(12)
memory usage: 28.0+ MB

```

```

[ ]: # Unindo dataset de score com dataset principal
df_merged_final = pd.merge(df, df_merged, left_on='user_id',
                             right_on='user_id', how='left')
df = df_merged_final

```

```

[ ]: # Validando
df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 111217 entries, 0 to 111216
Data columns (total 36 columns):
#   Column                Non-Null Count  Dtype
---  -
0   tweet_id              111217 non-null  object
1   raw_tweet             111217 non-null  object
2   preprocessed_tweet    111193 non-null  object
3   urls                  17025 non-null   object
4   hashtags              111217 non-null  object

```

```

5  mentions          111217 non-null object
6  emojis            111217 non-null object
7  is_retweet        111217 non-null int64
8  original_author   69620 non-null object
9  is_truncated       111217 non-null int64
10 is_upper          111217 non-null int64
11 tweet_length      111217 non-null int64
12 has_keywords      111217 non-null int64
13 tweet_created_at  111217 non-null datetime64[ns]
14 reply_to_tweet_id 28331 non-null object
15 reply_to_user_id  28535 non-null object
16 retweets          111217 non-null int64
17 likes             111217 non-null int64
18 engagement_raw    111217 non-null int64
19 engagement_rate    111217 non-null float64
20 engagement_index  111217 non-null float64
21 url_count         111217 non-null int64
22 hashtags_count    111217 non-null int64
23 mentions_count    111217 non-null int64
24 emoji_count       111217 non-null int64
25 user_id           111217 non-null object
26 followers_count   111217 non-null int64
27 user_created_at   111217 non-null datetime64[ns]
28 is_verified       111217 non-null int64
29 account_age_days  111217 non-null int64
30 collection_date   111217 non-null datetime64[ns]
31 is_withheld       111217 non-null int64
32 electoral_phase   111217 non-null object
33 bot_score_heuristica 111217 non-null float64
34 bot_score_botometer 111002 non-null float64
35 date_calculated   111002 non-null datetime64[ns]
dtypes: datetime64[ns](4), float64(4), int64(16), object(12)
memory usage: 30.5+ MB

```

C.3.2 Calculando Score Final

O **score final** de cada conta será calculado como uma combinação ponderada entre:

- **Score Heurístico** (10% de peso) – baseado em regras de classificação customizadas.
- **Score Botometer X** (90% de peso) – obtido da API.

A fórmula é:

$$Score_{Final} = (0.9 \times Score_{Botometer} \times Confiabilidade) + (0.1 \times Score_{Heurístico})$$

Como o *Botometer X* utiliza dados pré-calculados (até maio de 2023), um **fator de confiabilidade** será aplicado para penalizar scores baseados em *tweets* distantes do período eleitoral de 2022. Esse fator segue um **decaimento exponencial**:

$$\text{Confiabilidade} = e^{-k \cdot \text{DistânciaMédia}}$$

- **Distância Média:** média dos dias entre a data de cálculo e os *tweets* analisados.
- **k:** constante de decaimento que define a penalização temporal.

A **meia-vida** ($t_{1/2}$) é o tempo necessário para que a confiabilidade caia pela metade, e está relacionada à constante (k) por:

$$t_{1/2} = \frac{\ln(2)}{k}$$

No contexto da pesquisa, definir uma meia-vida permite calibrar intuitivamente a penalização. Por exemplo, uma meia-vida de 180 dias implica que a confiabilidade de um conjunto de *tweets* cairá para 50% após cerca de seis meses de diferença temporal.

Referências:

<https://www.gauthmath.com/knowledge/What-is-exponential-decay--7389727628099764234>

<https://www.geeksforgeeks.org/maths/real-world-examples-of-exponential-decay/>

<https://mathcracker.com/pt/formula-decaimento-exponencial>

A seguir, será definido empiricamente o valor da meia-vida com base na distribuição de `date_calculated`, de forma a refletir a concentração temporal dos dados.

```
[ ]: # Calcula a diferença em dias
time_difference_days = (df['tweet_created_at'].min() - df['date_calculated'].
    ↪min()).days

# Converte a diferença em dias para anos
time_difference_years = time_difference_days / 365

print(f"Diferença aproximada em dias: {time_difference_days:.0f}")
print(f"Diferença aproximada em anos: {time_difference_years:.0f}")
```

Diferença aproximada em dias: 3085

Diferença aproximada em anos: 8

```
[ ]: # Agrupamento por ano
df.groupby(df['date_calculated'].dt.year).size()
```

```
[ ]: date_calculated
2014.0      2
2015.0      5
2016.0      2
2017.0      1
2018.0      6
2019.0      3
2020.0     19
2021.0     52
2022.0    13752
```

```
2023.0    97160
dtype: int64
```

A maior parte dos scores foi calculada em período próximo ao intervalo de interesse (2022–2023), embora existam registros significativamente mais antigos. Uma escolha adequada de meia-vida deve manter alta confiabilidade para dados temporalmente próximos, ao mesmo tempo em que preserva, com peso reduzido, a contribuição de observações mais distantes no tempo.

```
[ ]: # Plotar a curva baseado na meia-vida informada
def plot_curva_meia_vida(meia_vida, dias_max):
    dias = np.arange(0, dias_max + 1)
    confiabilidade = 0.5 ** (dias / meia_vida)

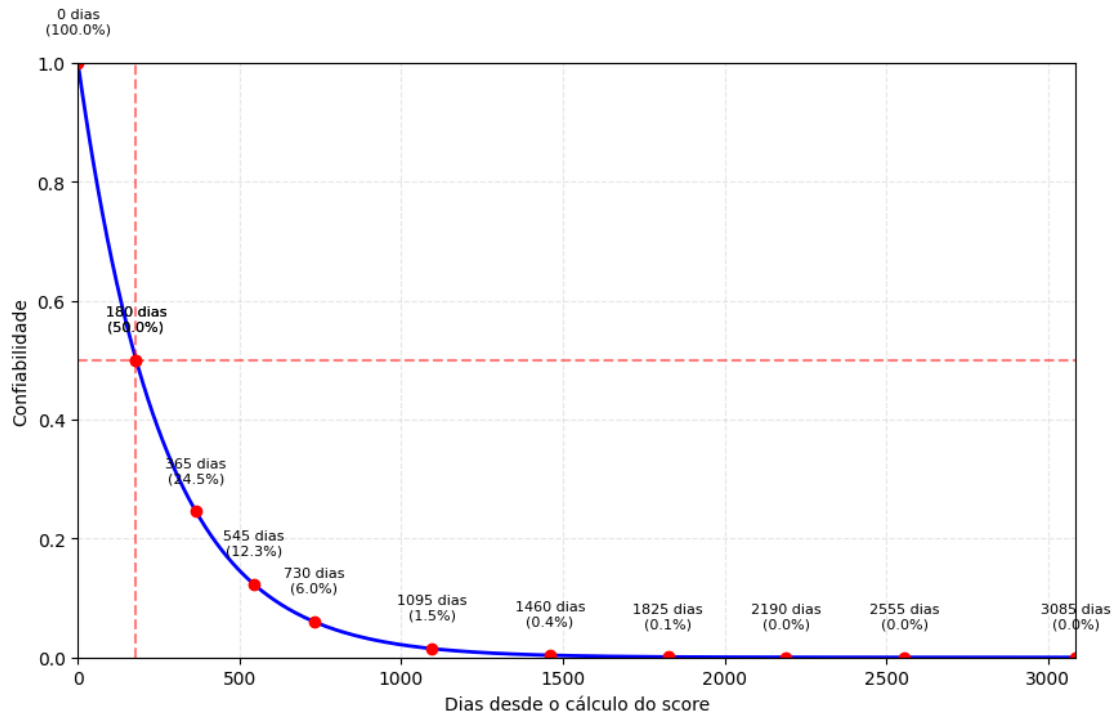
    plt.figure(figsize=(10, 6))
    plt.plot(dias, confiabilidade, 'b-', linewidth=2)

    # Destacar pontos-chave
    pontos = [meia_vida, 0, 180, 365, 545, 730, 1095, 1460, 1825, 2190, 2555,
    ↪ dias_max ]
    for d in pontos:
        if d <= dias_max:
            conf = 0.5 ** (d / meia_vida)
            plt.plot(d, conf, 'ro')
            plt.text(d, conf + 0.05, f'{d} dias\n({conf:.1%})', ha='center',
    ↪ size=8 )

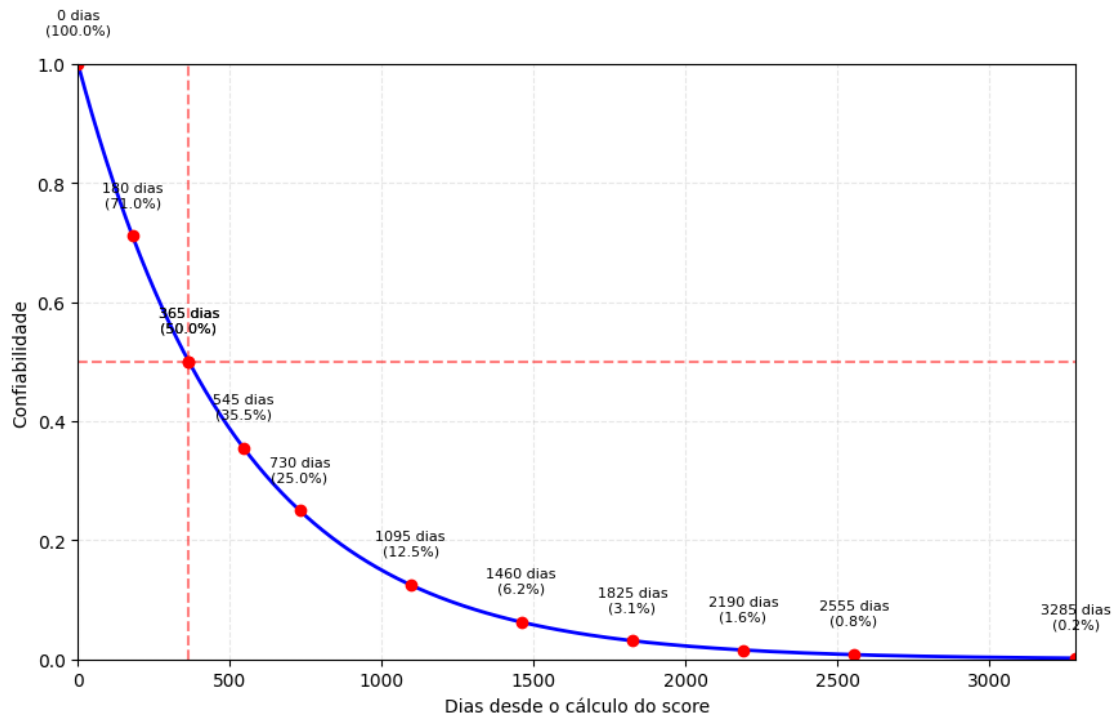
    plt.axhline(y=0.5, color='r', linestyle='--', alpha=0.5)
    plt.axvline(x=meia_vida, color='r', linestyle='--', alpha=0.5)

    plt.xlabel('Dias desde o cálculo do score')
    plt.ylabel('Confiabilidade')
    plt.grid(True, linestyle='--', alpha=0.3)
    plt.ylim(0, 1)
    plt.xlim(0, dias_max)
    plt.show()

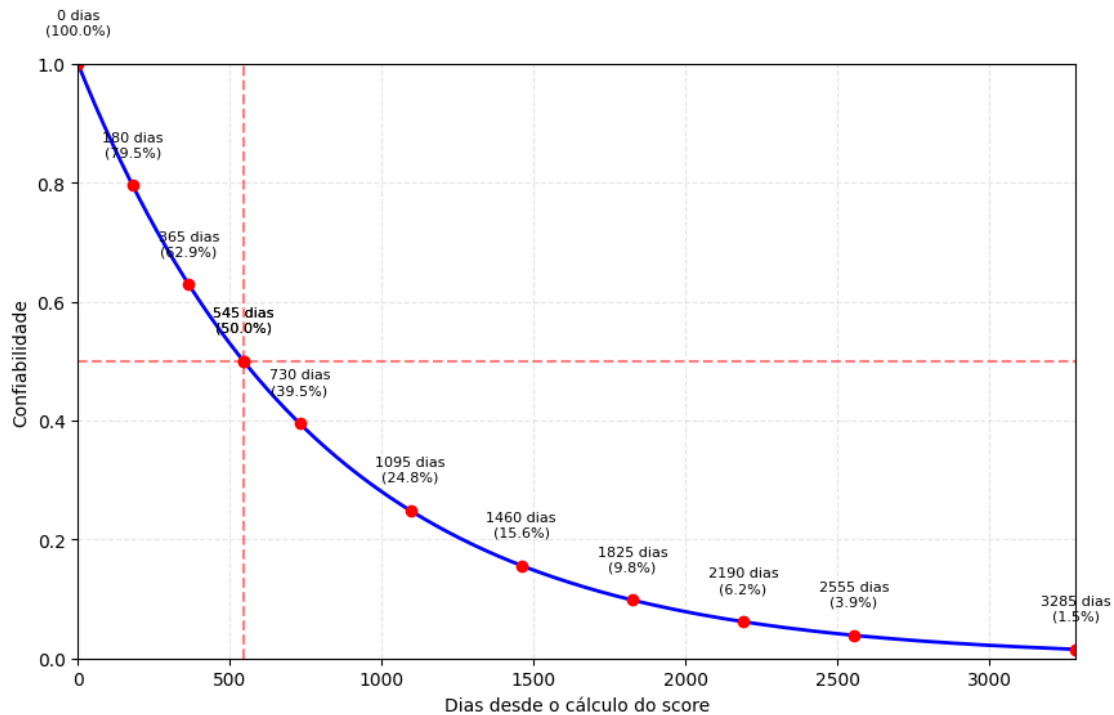
[ ]: # Gerar gráfico meia vida em 180 dias
plot_curva_meia_vida(meia_vida=180, dias_max=3085)
```



```
[ ]: # Gerar gráfico meia vida em 365 dias
plot_curva_meia_vida(meia_vida=365, dias_max=3285)
```



```
[ ]: # Gerar gráfico meia vida em 545 dias
plot_curva_meia_vida(meia_vida=545, dias_max=3285)
```



Dentre as opções avaliadas, adota-se meia-vida de 365 dias (um ano), por preservar o peso do período de interesse (2022–início de 2023) e ao mesmo tempo penalizar de forma progressiva observações temporalmente distantes.

```
[ ]: # Configuração clara da meia-vida (em dias)
HALF_LIFE = 365 # Dias para confiabilidade cair 50%
lambda_decay = np.log(2) / HALF_LIFE
```

```
[ ]: lambda_decay
```

```
[ ]: np.float64(0.0018990333713971104)
```

```
[ ]: def calcular_score_final(score_heuristica, score_botometer, date_calculated,
    ↪ user_tweet_dates):

    # Converter datas
    data_calculo = datetime.datetime.strptime(date_calculated, '%Y-%m-%d').
    ↪ date()
```

```

if not user_tweet_dates:
    return score_heuristica, 0.0, float('inf')

# Converter datas dos tweets
tweet_dates = [datetime.datetime.strptime(d, '%Y-%m-%d').date() for d in user_tweet_dates]

# Calcular distância temporal média
distancias = []
for data_tweet in tweet_dates:
    # Distância absoluta (independente de ser antes ou depois)
    distancia = abs((data_calculo - data_tweet).days)
    distancias.append(distancia)

distancia_media = round(np.mean(distancias), 2)

# Calcular fator de confiabilidade baseado na distância média
confiabilidade = round(np.exp(-lambda_decay * distancia_media), 2)

# Peso do Botometer e da Heurística
peso_botometer = 0.9
peso_heuristica = 0.1

# Calcular score final
score_bruto = ((peso_botometer * score_botometer) * confiabilidade) + (peso_heuristica * score_heuristica)
score_final = round(score_bruto, 2)

return score_final, confiabilidade, distancia_media, peso_botometer, peso_heuristica

```

```

[ ]: # Função para classificação
def classificar_conta(score_final):
    if score_final >= 0.7:
        return "Alto"
    elif score_final >= 0.4:
        return "Médio"
    else:
        return "Baixo"

```

```

[ ]: # Agrupar por user_id e coletar todas as datas de tweets
user_tweet_dates_dict = df.groupby('user_id')['tweet_created_at'].apply(
    lambda x: [d.strftime('%Y-%m-%d') for d in x]
).to_dict()

```

```

[ ]: len(user_tweet_dates_dict)

```

```
[ ]: 84693
```

```
[ ]: # Selecionar apenas colunas relevantes e remover duplicatas por user_id
df_users = df[['user_id', 'bot_score_heuristica',
               'bot_score_botometer',
               'date_calculated']].drop_duplicates(subset='user_id')

# Remover linhas com dados ausentes
df_users = df_users.dropna(subset=['bot_score_botometer', 'date_calculated'])
```

```
[ ]: # Aplicar as funções em cada linha
def aplicar_scores(row):
    user_id = row['user_id']
    score_heuristica = row['bot_score_heuristica']
    score_botometer = row['bot_score_botometer']
    date_calculated = row['date_calculated'].strftime('%Y-%m-%d')
    tweet_dates = user_tweet_dates_dict.get(user_id, [])

    try:
        score_final, confiabilidade, distancia_media, _, _ = \
        calcular_score_final(
            score_heuristica, score_botometer, date_calculated, tweet_dates
        )
        classificacao = classificar_conta(score_final)
    except Exception as e:
        score_final, confiabilidade, distancia_media, classificacao = None, \
        None, None, f"ERRO: {str(e)}"

    return pd.Series({
        'score_final': score_final,
        'reliability': confiabilidade,
        'average_distance': distancia_media,
        'automation_level': classificacao
    })

# Aplicar
df_users[['score_final', 'reliability',
          'average_distance',
          'automation_level']] = df_users.apply(aplicar_scores, axis=1)
```

```
[ ]: df_users
```

```
[ ]:
```

	user_id	bot_score_heuristica	bot_score_botometer	\
0	@USER_EDABE8C22D26	0.00	0.01	
1	@USER_D7E64A0A4978	0.00	0.47	
2	@USER_F31E8D4461E0	0.00	0.14	
3	@USER_8BB74A523D34	0.00	0.04	

4	@USER_F83A7D98B205	0.00	0.36
...	...	...	...
111212	@USER_AE9294E8071C	0.07	0.02
111213	@USER_03CCD9312874	0.07	0.36
111214	@USER_7FFC6EFAAEE6	0.00	0.28
111215	@USER_920DA341CEA1	0.00	0.45
111216	@USER_1E85F6EB7FFB	0.07	0.44

	date_calculated	score_final	reliability	average_distance	\
0	2023-05-27 22:27:20	0.01	0.60	270.0	
1	2023-04-06 04:35:47	0.28	0.66	219.0	
2	2023-05-19 18:44:50	0.08	0.61	262.0	
3	2023-05-14 00:35:14	0.02	0.61	257.0	
4	2023-05-30 12:32:45	0.19	0.60	273.0	
...	...	...	...	...	
111212	2023-04-23 13:32:31	0.02	0.76	142.0	
111213	2023-05-30 09:18:29	0.24	0.71	179.0	
111214	2023-05-22 13:18:34	0.18	0.72	171.0	
111215	2023-03-06 11:16:40	0.34	0.84	94.0	
111216	2022-12-30 18:36:29	0.38	0.95	28.0	

	automation_level
0	Baixo
1	Baixo
2	Baixo
3	Baixo
4	Baixo
...	...
111212	Baixo
111213	Baixo
111214	Baixo
111215	Baixo
111216	Baixo

[84485 rows x 8 columns]

```
[ ]: df = df.merge(df_users[['user_id', 'score_final', 'reliability',
↪ 'average_distance', 'automation_level']], on='user_id', how='left')
```

```
[ ]: # Verificando usuários sem classificação (Deve ser 208)
df[df['automation_level'].isna()] \
    .groupby('user_id') \
    .size() \
    .reset_index(name='qtd') \
    .sort_values(by='qtd', ascending=False)
```

```
[ ]:          user_id  qtd
28  @USER_235EBD68F56D    3
3   @USER_01A479AB54E9    2
123 @USER_9A8851A5AAB3    2
84  @USER_63CDEB942994    2
154 @USER_BF3CDAC573FE    2
..   ...
203 @USER_F7D0A0CDCD83    1
204 @USER_F87C3774102C    1
205 @USER_F99A223662A0    1
206 @USER_FA802F724DF2    1
207 @USER_FB5564032847    1
```

[208 rows x 2 columns]

```
[ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 111217 entries, 0 to 111216
Data columns (total 40 columns):
#   Column                Non-Null Count  Dtype
---  -
0   tweet_id              111217 non-null  object
1   raw_tweet             111217 non-null  object
2   preprocessed_tweet    111193 non-null  object
3   urls                  17025 non-null   object
4   hashtags              111217 non-null  object
5   mentions              111217 non-null  object
6   emojis                111217 non-null  object
7   is_retweet            111217 non-null  int64
8   original_author       69620 non-null   object
9   is_truncated          111217 non-null  int64
10  is_upper              111217 non-null  int64
11  tweet_length          111217 non-null  int64
12  has_keywords          111217 non-null  int64
13  tweet_created_at      111217 non-null  datetime64[ns]
14  reply_to_tweet_id     28331 non-null   object
15  reply_to_user_id      28535 non-null   object
16  retweets              111217 non-null  int64
17  likes                 111217 non-null  int64
18  engagement_raw        111217 non-null  int64
19  engagement_rate       111217 non-null  float64
20  engagement_index      111217 non-null  float64
21  url_count             111217 non-null  int64
22  hashtags_count        111217 non-null  int64
23  mentions_count        111217 non-null  int64
24  emoji_count           111217 non-null  int64
25  user_id               111217 non-null  object
```

```
26 followers_count      111217 non-null int64
27 user_created_at      111217 non-null datetime64[ns]
28 is_verified          111217 non-null int64
29 account_age_days     111217 non-null int64
30 collection_date      111217 non-null datetime64[ns]
31 is_withheld          111217 non-null int64
32 electoral_phase      111217 non-null object
33 bot_score_heuristica 111217 non-null float64
34 bot_score_botometer  111002 non-null float64
35 date_calculated      111002 non-null datetime64[ns]
36 score_final          111002 non-null float64
37 reliability          111002 non-null float64
38 average_distance     111002 non-null float64
39 automation_level     111002 non-null object
dtypes: datetime64[ns](4), float64(7), int64(16), object(13)
memory usage: 33.9+ MB
```

```
[ ]: # Salvando
df.to_csv("/content/drive/MyDrive/TCC/Data/TUEBR22.csv", index=False)
```